

Merge as Graph Extension

Sasson Margalio

July 2026

Abstract

This paper proposes a graph-theoretic interpretation of Merge inside the Workspace. The Workspace is represented as a directed graph, and syntactic objects are represented as Paths or subgraphs inside that graph. An application of Merge preserves the Paths already present and adds a new accessible Path. The new Path can then participate in later applications of Merge inside the same accumulated Workspace.

The proposal is stated inside a seven-Layer architecture, with C-I and A-P as external systems and five internal Layers of the Language Faculty: Semantic Component (Σ), A-Syntax, A-bar Syntax, Distributed Morphology (DM), and Phonological Component (Φ). Each internal Layer applies an ordered grammar of Extension Rules through Expand, constructs a new Output Graph by Relinearization, and transfers that graph to the next Layer. The Layers are traversed in one order in *generation* and in the reverse order in *comprehension*, while the same general Expand procedure applies across the internal Layers. The grammars differ from Layer to Layer and by direction of traversal, but the formal processing cycle remains the same.

1. The Seven-Layer Architecture

I assume seven Layers in the overall architecture. Two of them are external systems:

- C-I,
- A-P.

The other five are internal Layers of the Language Faculty:

- Semantic Component (Σ),
- A-Syntax,
- A-bar Syntax,
- Distributed Morphology (DM),
- Phonological Component (Φ).

The seven Layers together form the architecture assumed here. C-I is the intelligent Agent itself, interacting with other cognitive systems. A-P is the biological articulatory-perceptual performance system. Between them lie the five internal Layers that constitute the linguistic machinery of the Language Faculty.

The full *generation* pipeline is:

(1)

C-I $\rightarrow$ [Agentic Form] $\rightarrow$
 Semantic Component (Σ) $\rightarrow$ [Event Structure] $\rightarrow$
 A-Syntax $\rightarrow$ [NP Structure] $\rightarrow$
 A-bar Syntax $\rightarrow$ [Surface Structure] $\rightarrow$
 Distributed Morphology (DM) $\rightarrow$ [Phonological Representation] $\rightarrow$
 Phonological Component (Φ) $\rightarrow$ [Phonetic Form] $\rightarrow$
 A-P

The bracketed terms name representations transferred between adjacent Layers.

In *generation*, the Layers are traversed in the following order:

(2) C-I $\rightarrow \Sigma \rightarrow$ A-Syntax $\rightarrow$ A-bar Syntax $\rightarrow$ DM $\rightarrow \Phi \rightarrow$ A-P

C-I supplies Agentic Form to the first internal Layer, the Semantic Component Σ . From that point on, each Layer receives the representation produced by the preceding Layer, applies its own grammar, and produces the representation required by the next Layer. At the end of the pipeline, the Phonological Component supplies Phonetic Form to A-P.

In *comprehension*, the same Layers are traversed in the reverse order:

(3) A-P $\rightarrow \Phi \rightarrow$ DM $\rightarrow$ A-bar Syntax $\rightarrow$ A-Syntax $\rightarrow \Sigma \rightarrow$ C-I

In this direction, the derivation begins with Phonetic Form supplied by A-P and moves inward toward C-I. *Comprehension* therefore does not require a second architecture. It

uses the same seven Layers and the same general processing cycle but traverses them in the opposite direction.

In standard linguistic notation, a rule such as

(4) $VP \Rightarrow V NP$

is naturally understood as a parsing rule: when a $V NP$ configuration is found, it may be recognized as a VP . The proposal developed here gives this kind of rule a graph-theoretic interpretation. An Extension Rule written as $Left \Rightarrow Right$ is applied by finding material that matches the Right side and adding material corresponding to the Left side.

This interpretation makes it possible to treat Merge as a graph-extension procedure. Merge preserves the material already present in the Workspace while adding structure that can participate in later applications of Merge.

Each Layer is associated with two grammars, one for *generation* and one for *comprehension*. Each grammar consists of an ordered set of Extension Rules. A Layer receives a Workspace, applies Merge repeatedly inside that Workspace, and then produces a new graph through Relinearization. Transfer passes that graph as the initial Workspace of the next Layer. The grammars differ from Layer to Layer, but the basic processing cycle remains the same.

The paper is organized as follows. Section 2 introduces the Workspace as a graph, and Section 3 specifies the relation between Layers, grammars, Expand, and the initial Workspace. The graph notation used in the paper is given in Section 4. Section 5 states the graph-extension interpretation of Merge. Section 6 gives a simple example of Merge as graph extension, followed by a larger example in Section 7. Section 8 describes Layer grammars and the Expand procedure. Section 9 turns to Relinearization and Transfer. Section 10 concludes.

2. The Workspace as a Graph

Chomsky (2019) uses the term Workspace for the collection of syntactic objects available during a derivation. The present proposal is related to that use of the term, but I give the Workspace a specific graph-theoretic interpretation.

In this formalization, syntactic objects are represented by Paths or subgraphs inside the Workspace graph. A Path is a sequence of nodes connected by directed Edges. A **Labeled**

Path is a subgraph consisting of a Path through Token Nodes, together with the Label Nodes associated with those Token Nodes and the Edges linking to them.

The **Label sequence** of a Labeled Path is the sequence of Labels carried by the Token Nodes along the Path. This sequence may match a Pattern, correspond to a Tag, or be neither. Pattern and Tag are terms used for the two sides of an Extension Rule; they will be defined below.

A Path selected for a particular application of an Extension Rule has an Entry Node and an Exit Node. The Entry Node is the first node of the selected Path, and the Exit Node is its last node. The **Source Set** consists of the nodes that immediately precede the Entry Node in the relevant Workspace paths. The **Goal Set** consists of the nodes that immediately follow the Exit Node in those paths.

Merge applies to objects available in the Workspace by adding a new Path in the position determined by the Source Set and the Goal Set of the Path already found. The resulting Workspace preserves the earlier material while extending the graph.

This interpretation preserves the familiar idea that Merge applies repeatedly and that each new object can participate in later computation. The difference is in the representation of the Workspace. Instead of reducing the derivation to a single current tree or object, the Workspace graph may contain several Paths in the same local position.

Earlier Paths therefore remain available in the graph, while newly added Paths can themselves become input to later applications of Merge. The Workspace contains the accumulated result of repeated applications of Merge.

3. Layers, Expand, and the Initial Workspace

A Layer is a persistent component of the architecture. The Workspace is the changing graph on which computation takes place.

Only the five internal Layers have grammars in the sense used here. These are Σ , A-Syntax, A-bar Syntax, DM, and Φ . Each internal Layer is associated with two grammars: one for *generation* and one for *comprehension*. A grammar consists of an ordered set of Extension Rules. The five internal Layers share the same general form of computation, but not the same rules.

The procedure that applies the grammar of an internal Layer is called Expand. Expand takes a Workspace as input and applies the rules of the selected grammar repeatedly, one

application of Merge at a time. Expand works inside the current Workspace. It extends that Workspace by adding Paths to it.

When no further rule application is possible, Relinearization begins. Relinearization first creates a precise full copy of the completed Workspace. It then applies its own procedures to that copy. The resulting graph is the Output Graph transferred to the next Layer.

The same Expand procedure is used in all five internal Layers. What differs is the grammar selected by the Layer for the current direction of traversal.

A Layer receives its Workspace from the preceding component. Before Expand begins, this Workspace must also contain the Labels expected by the grammar of the present Layer. Merge then works with a graph that already contains the relevant Label Nodes. It introduces new Token Nodes, links them to those Labels, and adds new Paths to the Workspace.

4. Nodes, Edges, Labels, and Paths

The Workspace is represented here as a directed graph, consisting of Nodes and directed Edges. A Node is written in the form:

(5) (id)

The unique key of a Node is its id. Within a single graph, there is at most one Node with the same id.

The graph does not use node classes. The distinction between Token Nodes and Label Nodes is structural. A Label Node is a Node with no outgoing Edges. A Token Node is a Node with one or more outgoing Edges.

In the examples below, the Token Node labelled begin is always written as (1), and the Token Node labelled end is always written as (2). Other Token Nodes are written with running numerical ids. To make Token Node ids easy to distinguish from example numbers, I use three-digit ids for ordinary Token Nodes:

(6)

a. (157)

b. (158)

c. (159)

Label Nodes are written with mnemonic ids:

(7)

- a. (begin)
- b. (end)
- c. (E)
- d. (F)

The Labels begin and end mark the beginning and end of a Workspace Path. They are ordinary Label Nodes.

A directed Edge is indicated by an arrow:

(8) (157) $\rightarrow$ (158)

A Path is a sequence of Nodes connected by directed Edges. For example:

(9) (157) $\rightarrow$ (158) $\rightarrow$ (159)

represents a Path through the Token Nodes (157), (158), and (159).

Labelling is also represented by directed Edges. Thus a Token Node associated with the Label E is represented by an Edge from the Token Node to the Label Node:

(10) (158) $\rightarrow$ (E)

A graph containing both a Path and its Labels can therefore be written explicitly as the following set of Edges:

(11)

- a. (1) $\rightarrow$ (begin)
- b. (2) $\rightarrow$ (end)
- c. (1) $\rightarrow$ (157)
- d. (157) $\rightarrow$ (158)
- e. (158) $\rightarrow$ (2)
- f. (157) $\rightarrow$ (E)
- g. (158) $\rightarrow$ (F)

For readability, I also use a compact curly-brace notation. In this notation, a Label may be written immediately after the Token Node that carries it:

(12) (158) {F}

This is an abbreviation for the Edge from the Token Node to the Label Node:

(13) (158) $\rightarrow$ (F)

Using this abbreviation, the graph in (11) can be written more compactly as:

(14) (1) {begin} → (157) {E} → (158) {F} → (2) {end}

The arrows in (14) give the main directed Path. The curly-brace expressions display the Labels carried by the Token Nodes on that Path.

5. Extension Rules and a Simple Merge Example

Extension Rules retain familiar linguistic notation:

(15) Tag => Pattern

The left side of a Rule is the **Tag** to be inserted. A Tag is a sequence of Labels. When the Tag contains one Label, Merge inserts a one-node Path whose Token Node carries that Label. When the Tag contains more than one Label, Merge inserts a Path whose Token Nodes carry the Labels of the Tag in order.

The right side of a Rule is the **Pattern**. A Pattern is a sequence of Labels to be found in the Label sequence of an already available Labeled Path.

Thus, a Rule such as:

(16) A => B C

is applied by finding a Labeled Path whose Label sequence is B C and inserting a new Path whose Label sequence is A.

Assume that the Workspace contains the following Path:

(17) (1) {begin} → (357) {B} → (358) {C} → (2) {end}

The Pattern B C is found on the following Labeled Path:

(18) [(357) {B} → (358) {C}]

The Entry Node of this Path is (357) {B}, and the Exit Node is (358) {C}. The Source Set contains (1) {begin}, and the Goal Set contains (2) {end}.

Merge inserts a fresh one-node Path whose Token Node carries the Label A:

(19) [(359) {A}]

The inserted Path is linked to the Source Set and the Goal Set:

(20) (1) {begin} → (359) {A} → (2) {end}

After this application of Merge, the Workspace contains both the original Path and the inserted Path:

(21)

- a. (1) {begin} → (357) {B} → (358) {C} → (2) {end}
- b. (1) {begin} → (359) {A} → (2) {end}

The original Path has not been erased. It remains in the Workspace, while the newly inserted Path makes the Label A available for later applications of Merge.

The Path that matched the Pattern is the **origin** of the inserted Path. In this example, the Labeled Path in (18) is the origin of the inserted Path in (19). Every Path inserted by Merge therefore has an origin: the Path that matched the Pattern in the Rule application that inserted it.

6. The Expand Procedure

Section 5 defined Extension Rules of the form Tag => Pattern. The Tag determines the Path to be inserted. The Pattern is matched against the Label sequence of an already available Labeled Path.

For a given internal Layer and direction of traversal, Expand proceeds as follows:

(51) Expand

- a. receive a Workspace;
- b. select the grammar of the present internal Layer for the current direction of traversal;
- c. collect the Labeled Paths in the Workspace that have not yet been checked;
- d. consider those Labeled Paths from shorter to longer;
- e. for each such Labeled Path, scan the Rules of the selected grammar in order;
- f. when the Pattern of a Rule matches the Label sequence of that Labeled Path, apply Merge;
- g. mark the Labeled Path as checked;
- h. repeat from (51c) until no unchecked Labeled Paths remain;
- i. return the expanded Workspace.

A Labeled Path may become available only after Merge has inserted new material. Such a Path is not considered in the same pass in which it is created. It is considered in a later pass, after Expand again collects the Labeled Paths that have not yet been checked.

In a single application of Merge, the Pattern of a Rule is found on a Labeled Path in the Workspace. The first Token Node of that Path is its Entry Node, and the last Token Node is

its Exit Node. The Source Set contains the Token Nodes that immediately precede the Entry Node. The Goal Set contains the Token Nodes that immediately follow the Exit Node.

Merge inserts a new Path corresponding to the Tag. Each Token Node in the Source Set is linked to the Entry Node of the inserted Path. The Exit Node of the inserted Path is linked to each Token Node in the Goal Set. The matched Path is not removed.

The same Rule is not applied twice to the same Labeled Path, because each Labeled Path is marked after it has been checked. This prevents Expand from repeatedly inserting the same Path in the same local position.

Thus, Expand does not replace one Path with another. It extends the Workspace by adding Paths. Newly inserted Paths may then make additional Patterns available for later Rule applications.

The grammar supplied to Expand differs from Layer to Layer and by direction of traversal. The procedure itself remains the same. In this sense, the five internal Layers share the same general computational form while differing in their rule inventories.

7. Merge Inside a Larger Workspace

The previous section used a minimal example. The same procedure applies inside a larger Workspace.

Consider the sentence:

(22) I read a book

The initial Workspace contains the following Path:

(23)

(1) {begin} → (157) {I} → (158) {read} → (159) {a} → (160) {book} → (2) {end}

Suppose the Layer grammar contains the following Rules.

(24) Rules

a. pronoun => I

b. verb => read

- c. Det => a
- d. noun => book
- e. DP => pronoun
- f. DP => Det noun
- g. VP => verb DP
- h. TP => DP VP

Rule (24a) finds the Labeled Path:

(25) [(157) {I}]

The Source Set contains (1) {begin}, and the Goal Set contains (158) {read}. Merge inserts the Path (161) {pronoun}:

(26) (1) {begin} → (161) {pronoun} → (158) {read}

The original Token Node (157) {I} is not removed. The Workspace now contains both the original word path through (157) and the inserted path through (161).

Rule (24b) finds the Labeled Path:

(27) [(158) {read}]

At this point, the Source Set contains (157) {I}, (161) {pronoun}, and the Goal Set contains (159) {a}. Merge inserts the Path (162) {verb}. One Path now available in the Workspace is:

(28)

(1) {begin} → (161) {pronoun} → (162) {verb} → (159) {a} → (160) {book} → (2) {end}

Rule (24c) finds the Labeled Path:

(29) [(159) {a}]

The Source Set contains (158) {read}, (162) {verb}, and the Goal Set contains (160) {book}. Merge inserts the Path (163) {Det}. One Path now available is:

(30)

(1) {begin} → (161) {pronoun} → (162) {verb} → (163) {Det} → (160) {book} → (2) {end}

Rule (24d) finds the Labeled Path:

(31) [(160) {book}]

The Source Set contains (159) {a}, (163) {Det}, and the Goal Set contains (2) {end}. Merge inserts the Path (164) {noun}. One Path now available is:

(32)

(1) {begin} → (161) {pronoun} → (162) {verb} → (163) {Det} → (164) {noun} → (2) {end}

The Workspace now contains the Labeled Path:

(33) [(161) {pronoun}]

Rule (24e) can therefore apply. For the Path in (33), the Source Set contains (1) {begin}, and the Goal Set contains (158) {read}, (162) {verb}. Merge inserts the Path (165) {DP}. One Path now available is:

(34)

(1) {begin} → (165) {DP} → (162) {verb} → (163) {Det} → (164) {noun} → (2) {end}

The Workspace also contains the Labeled Path:

(35) [(163) {Det} → (164) {noun}]

Rule (24f) can therefore apply. For the Path in (35), the Source Set contains (158) {read}, (162) {verb}, and the Goal Set contains (2) {end}. Merge inserts the Path (166) {DP}. One Path now available is:

(36)

(1) {begin} → (165) {DP} → (162) {verb} → (166) {DP} → (2) {end}

The Workspace now contains the Labeled Path:

(37) [(162) {verb} → (166) {DP}]

Rule (24g) can therefore apply. For the Path in (37), the Source Set contains (157) {I}, (161) {pronoun}, (165) {DP}, and the Goal Set contains (2) {end}. Merge inserts the Path (167) {VP}. One Path now available is:

(38)

(1) {begin} → (165) {DP} → (167) {VP} → (2) {end}

The Workspace now contains the Labeled Path:

(39) [(165) {DP} → (167) {VP}]

Rule (24h) can therefore apply. For the Path in (39), the Source Set contains (1) {begin}, and the Goal Set contains (2) {end}. Merge inserts the Path (168) {TP}. The Workspace now contains the Path:

(40)

(1) {begin} → (168) {TP} → (2) {end}

This example shows Merge applying inside a Workspace that contains more than one word. The computation begins from the concrete input Path. Lexical category Paths are added first, and phrasal Paths are added when the relevant Labeled Paths become available. Expand continues until no further Rule in the selected grammar can apply.

8. The Expanded Workspace

The derivation in Section 7 can also be presented as the graph produced by Expand. The expanded Workspace is shown in (41)–(67).

The structural Labels are given first.

(41) Structural labels

- a. (1) {begin}
- b. (2) {end}

The following Labels are used in the expanded Workspace.

(42) Labels used in the expanded Workspace

- a. (I)
- b. (read)
- c. (a)
- d. (book)
- e. (pronoun)
- f. (verb)
- g. (Det)
- h. (noun)
- i. (DP)
- j. (VP)
- k. (TP)

The Rules are listed in (43)–(50).

(43) pronoun => I

(44) verb => read

(45) Det => a

(46) noun => book

(47) DP => pronoun

(48) DP => Det noun

(49) VP => verb DP

(50) TP => DP VP

The original input Path is shown in (51).

(51) Input Path

- a. (1) {begin} → (157) {I}
- b. (157) {I} → (158) {read}
- c. (158) {read} → (159) {a}
- d. (159) {a} → (160) {book}
- e. (160) {book} → (2) {end}

Rule (43) applies first to the initial word I. The matched Path appears in (52), and Merge adds the corresponding pronoun Path in the same local position, as shown in (53).

(52) Path matched by Rule 43

- a. (1) {begin} → (157) {I}
- b. [(157) {I}]
- c. (157) {I} → (158) {read}

(53) Path inserted by Rule 43

- a. (1) {begin} → (161) {pronoun}
- b. (161) {pronoun}
- c. (161) {pronoun} → (158) {read}

The new pronoun Path now gives the graph an additional route into the position before read. Rule (44) uses the original read Path in (54) and inserts the verb Path in (55).

(54) Path matched by Rule 44

- a. (157) {I}, (161) {pronoun} → (158) {read}
- b. [(158) {read}]
- c. (158) {read} → (159) {a}

(55) Path inserted by Rule 44

- a. (157) {I}, (161) {pronoun} → (162) {verb}
- b. (162) {verb}
- c. (162) {verb} → (159) {a}

The same process continues at the next word. Rule (45) reads a in the context already created by the earlier Paths and adds the determiner Path displayed in (57).

(56) Path matched by Rule 45

- a. (158) {read}, (162) {verb} → (159) {a}
- b. [(159) {a}]
- c. (159) {a} → (160) {book}

(57) Path inserted by Rule 45

- a. (158) {read}, (162) {verb} → (163) {Det}
- b. (163) {Det}
- c. (163) {Det} → (160) {book}

Rule (46) then applies to book. The match in (58) places book immediately before the final boundary, and the inserted noun Path in (59) occupies the same position.

(58) Path matched by Rule 46

- a. (159) {a}, (163) {Det} → (160) {book}
- b. [(160) {book}]
- c. (160) {book} → (2) {end}

(59) Path inserted by Rule 46

- a. (159) {a}, (163) {Det} → (164) {noun}
- b. (164) {noun}
- c. (164) {noun} → (2) {end}

At this point the graph contains a pronoun Path. Rule (47) uses it to form the subject DP; the relevant match is (60), and the new DP Path is (61).

(60) Path matched by Rule 47

- a. (1) {begin} → (161) {pronoun}
- b. [(161) {pronoun}]
- c. (161) {pronoun} → (158) {read}, (162) {verb}

(61) Path inserted by Rule 47

- a. (1) {begin} → (165) {DP}
- b. (165) {DP}
- c. (165) {DP} → (158) {read}, (162) {verb}

The object DP is built differently. Rule (48) does not apply to a one-node Path, but to the two-node sequence Det noun. This sequence is shown in (62), and the inserted object DP appears in (63).

(62) Path matched by Rule 48

- a. (158) {read}, (162) {verb} → (163) {Det}

- b. [(163) {Det} → (164) {noun}]
- c. (164) {noun} → (2) {end}

(63) Path inserted by Rule 48

- a. (158) {read}, (162) {verb} → (166) {DP}
- b. (166) {DP}
- c. (166) {DP} → (2) {end}

Once the verb and object DP Paths are both available, Rule (49) can form the VP. The match in (64) spans those two Paths, and the resulting VP Path is inserted in (65).

(64) Path matched by Rule 49

- a. (157) {I}, (161) {pronoun}, (165) {DP} → (162) {verb}
- b. [(162) {verb} → (166) {DP}]
- c. (166) {DP} → (2) {end}

(65) Path inserted by Rule 49

- a. (157) {I}, (161) {pronoun}, (165) {DP} → (167) {VP}
- b. (167) {VP}
- c. (167) {VP} → (2) {end}

The final application combines the subject DP and the VP. Rule (50) matches the Path in (66) and adds the TP Path in (67).

(66) Path matched by Rule 50

- a. (1) {begin} → (165) {DP}
- b. [(165) {DP} → (167) {VP}]
- c. (167) {VP} → (2) {end}

(67) Path inserted by Rule 50

- a. (1) {begin} → (168) {TP}
- b. (168) {TP}
- c. (168) {TP} → (2) {end}

9. Relinearization and Transfer

The procedure described in Section 6 returns the expanded Workspace. Relinearization begins only after that procedure has completed and no further rule application is possible.

Relinearization constructs the output graph for the next Layer. It creates a precise full copy of the completed Workspace; the completed Workspace itself is not changed. It then adds any missing label nodes for labels supported by the next Layer. The structural labels (begin) and (end) are supported by every Layer.

The filtering step removes from the copied graph all label nodes for labels not supported by the next Layer. When a node is removed from the graph, it is removed together with all incoming and outgoing edges attached to it.

Token nodes are not removed one by one. They are removed only through the removal of paths that are not retained in the output graph. The relevant paths are paths inserted by Merge and paths matched by rules that led to those insertions.

A path is licensed for the next Layer if, after the filtering step, it has no token node without a label. A path inserted by Merge is retained in the output graph only if it is licensed. A path matched by a rule is copied after a path inserted by Merge only if the path matched by that rule is licensed.

Streamline considers paths inserted by Merge from shorter to longer. Each path inserted by Merge is associated with the path matched by the rule application that inserted it. If both paths are licensed, Streamline adds a copy of the path matched by the rule at the end of the path inserted by Merge. The copy is placed between the path inserted by Merge and the nodes in its goal set. The edges from the former exit node of the path inserted by Merge to the goal set are replaced by edges from the new exit node, which is the last token node of the added copy.

Consider again the graph produced in Sections 7 and 8 for the sentence I read a book. Assume that the next Layer supports the labels (TP), (DP), (VP), (pronoun), (verb), (Det), (noun), (read), and (book), but not the labels (I) and (a). The filtering step removes the label nodes (I) and (a).

Rule (43) inserted the pronoun path by Merge. The path matched by rule (43) is not licensed, because the Label (I) is not supported by the next Layer. The pronoun path is licensed, so it is retained without a copy of the path matched by rule (43).

(68) Streamline for rule (43)

- a. (1) {begin} → (161) {pronoun}
- b. [(157)]
- c. (161) {pronoun} → (158) {read}

Rule (44) inserted the verb path by Merge. The path matched by rule (44) is licensed, and the verb path is licensed as well. Streamline therefore adds a copy of the matched path after the verb path.

(69) Streamline for rule (44)

- a. (157), (161) {pronoun} → (162) {verb}
- b. [(162) {verb} → (158) {read}]
- c. (158) {read} → (159)

Rule (45) inserted the Det path by Merge. The path matched by rule (45) is not licensed, because the Label (a) is not supported by the next Layer. The Det path is licensed, so it is retained without a copy of the path matched by rule (45).

(70) Streamline for rule (45)

- a. (158) {read}, (162) {verb} → (163) {Det}
- b. [(159)]
- c. (163) {Det} → (160) {book}

Rule (46) inserted the noun path by Merge. The path matched by rule (46) is licensed, and the noun path is licensed as well. Streamline therefore adds a copy of the matched path after the noun path.

(71) Streamline for rule (46)

- a. (159), (163) {Det} → (164) {noun}
- b. [(164) {noun} → (160) {book}]
- c. (160) {book} → (2) {end}

Rule (47) inserted the subject DP path by Merge. The path matched by rule (47) is licensed, and the subject DP path is licensed. Streamline adds the matched path after the subject DP path.

(72) Streamline for rule (47)

- a. (1) {begin} → (165) {DP}
- b. [(165) {DP} → (161) {pronoun}]
- c. (161) {pronoun} → (158) {read}, (162) {verb}

Rule (48) inserted the object DP path by Merge. The path matched by rule (48) is licensed, and the object DP path is licensed. Since the noun path has already been streamlined together with book, the result includes book after noun.

(73) Streamline for rule (48)

- a. (158) {read}, (162) {verb} → (166) {DP}

- b. [(166) {DP} → (163) {Det} → (164) {noun} → (160) {book}]
- c. (160) {book} → (2) {end}

Rule (49) inserted the VP path by Merge. The path matched by rule (49) is licensed, and the VP path is licensed. Streamline adds the already streamlined content of the matched path after the VP path.

(74) Streamline for rule (49)

- a. (157), (161) {pronoun}, (165) {DP} → (167) {VP}
- b. [(167) {VP} → (162) {verb} → ... → (164) {noun} → (160) {book}]
- c. (160) {book} → (2) {end}

Rule (50) inserted the TP path by Merge. The path matched by rule (50) is licensed, and the TP path is licensed. Streamline therefore adds the already streamlined content of the matched path after the TP path.

(75) Streamline for rule (50)

- a. (1) {begin} → (168) {TP}
- b. [(168) {TP} → (165) {DP} → ... → (164) {noun} → (160) {book}]
- c. (160) {book} → (2) {end}

The output graph includes the following full path:

(76) TP

- a. (1) {begin} → (168) {TP} → (165) {DP} → (161) {pronoun} → (167) {VP} → (162) {verb} → (158) {read} → (166) {DP} → (163) {Det} → (164) {noun} → (160) {book} → (2) {end}

Only token nodes on paths from (1) {begin} to (2) {end} are passed to the next Layer.

Transfer passes the output graph of one Layer as the initial Workspace of the next Layer. Thus, the procedure of Section 6 builds the completed Workspace, relinearization copies it and applies the filtering step and Streamline, and Transfer passes the resulting graph forward.

10. Conclusion

This paper proposes a formal interpretation of Merge as graph extension inside the Workspace. The Workspace is represented as a directed graph, and Merge adds a new accessible Path, without erasing the Paths already present.

The same general processing cycle is proposed for the five internal Layers of the Language Faculty: Semantic Component (Σ), A-Syntax, A-bar Syntax, Distributed Morphology (DM),

and Phonological Component (Φ). Each internal Layer is associated with two grammars, one for *generation* and one for *comprehension*. Each grammar consists of an ordered set of Extension Rules.

These Layers are traversed in one order in *generation* and in the reverse order in *comprehension*. The grammars differ from Layer to Layer and by direction of traversal, but the processing cycle remains the same. In both directions, a Layer receives a Workspace, applies the relevant ordered set of Extension Rules through Expand, constructs a new output graph by Relinearization, and transfers that graph to the next Layer. Merge is therefore treated as a path-adding step inside an accumulating Workspace graph.

References

Chomsky, Noam. 2019. The UCLA lectures (April 29–May 2, 2019). Transcribed and edited by Robert Freidin. LingBuzz/005485.

Chomsky, Noam, Ángel J. Gallego, and Dennis Ott. 2019. Generative grammar and the faculty of language: Insights, questions, and challenges. Catalan Journal of Linguistics, Special Issue: Generative Syntax: Questions, Crossroads, and Challenges, 229–261.

Halle, Morris, and Alec Marantz. 1993. Distributed Morphology and the pieces of inflection. In The view from Building 20: Essays in honor of Sylvain Bromberger, ed. Kenneth Hale and Samuel Jay Keyser, 111–176. Cambridge, MA: MIT Press.

Margaliot, Sasson. 2017. Minimalist semantics and pre-linguistic intelligence. LingBuzz/003409.